PROFILE



# 염동섭

**게임 클라이언트 프로그래머**

C++ · Unreal Engine 5.4

CONTACT

dsyeom931008@gmail.com

010-7181-7394

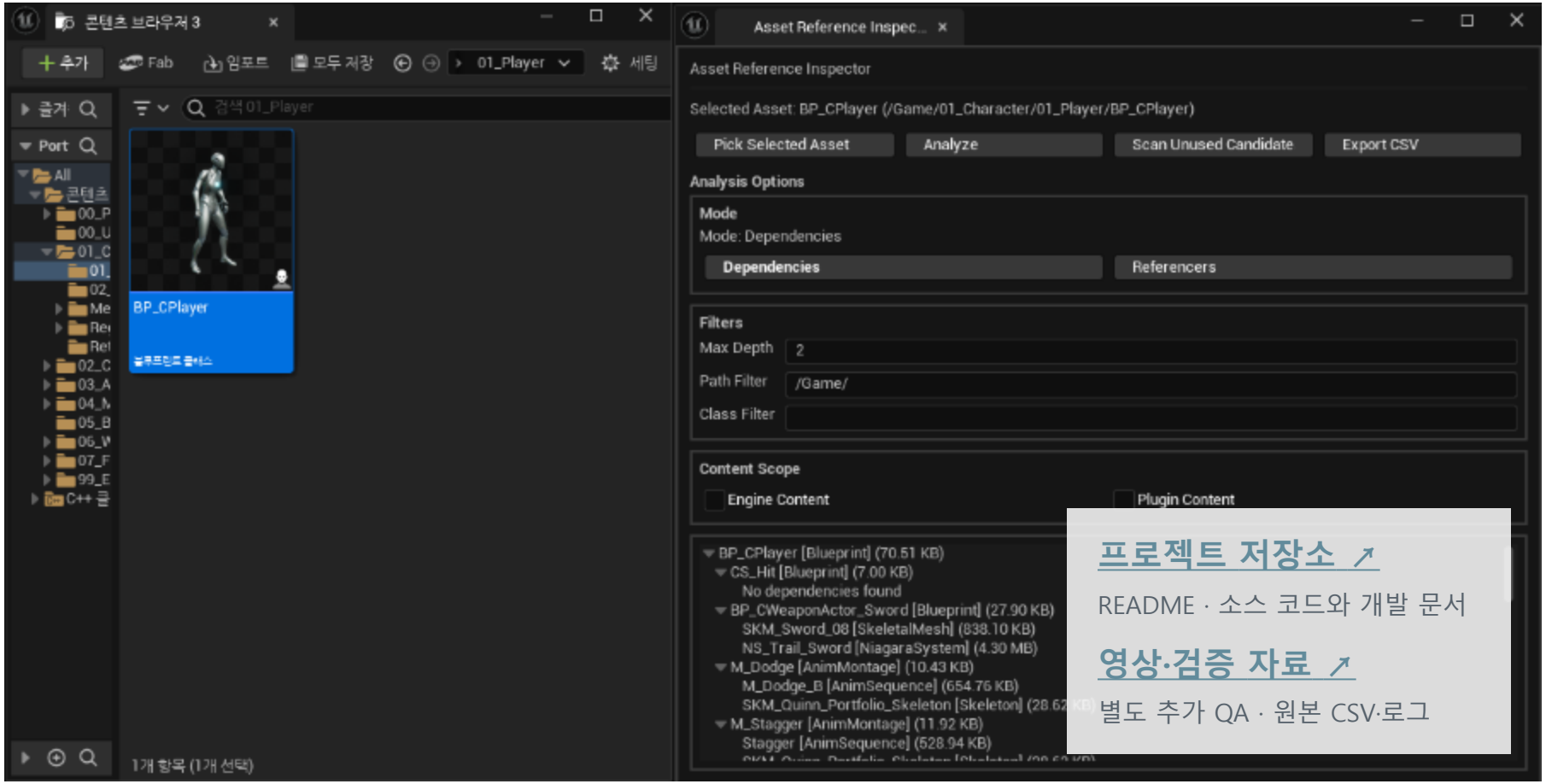
GitHub 프로필 ↗




Asset Reference Inspector

# AI 협업으로 개발한 에셋 참조 분석 플러그인

UE5.4 프로젝트의 에셋 참조 구조를 분석하는 독립 에디터 플러그인
에디터 도구 제작을 학습하고 실제 액션 프로젝트에 적용한 개발 사례




개발 당시 실제 액션 프로젝트 적용 화면

**프로젝트 저장소 ↗**

README · 소스 코드와 개발 문서

**영상·검증 자료 ↗**

별도 추가 QA · 원본 CSV·로그

# 프로젝트 구성

항목을 선택하면 해당 설명 페이지로 이동합니다. 관련 영상과 근거는 각 페이지에서 확인할 수 있습니다.

## 프로젝트 소개 03–04

**프로젝트 개요 →** 03

개발 목적과 환경, 작업 범위를 소개합니다.

**실제 사용 흐름 →** 04

에셋 선택부터 분석·저장까지 보여줍니다.

## 문제 해결·검증 05–10

**필터 입력 오류 →** 05–07

Enter 뒤 사라지는 필터 값을 수정합니다.

**미사용 후보 탐색 재설계 →** 08–10

선택 관계 밖의 후보도 찾도록 변경합니다.

## 프로젝트 구현·검증 11–15

**AI 협업 사전 준비 →** 11

구현 전에 정한 작업 기준을 소개합니다.

**전체 처리 구조 →** 12

입력부터 분석·출력까지의 구조입니다.

**공유·순환 관계 처리 →** 13

공유는 유지하고 순환 탐색은 종료합니다.

**CSV 저장 기능 →** 14

분석 결과를 파일로 저장하는 기능입니다.

**AI 리뷰 활용 →** 15

코드 리뷰를 수정·확인에 연결한 사례입니다.

# 에셋 참조 관계를 조회·기록하는 에디터 도구

개발 목적: Unreal Editor 도구 제작 학습과 실제 프로젝트의 에셋 참조 구조 분석
적용 범위: 게임 코드에 의존하지 않는 독립 플러그인으로 개발하고 액션 프로젝트에 적용

| 개발 기간 | 개발 규모 | 개발 형태 | 개발 환경 |
| --- | --- | --- | --- |
| **2026.07.13–07.20** | **1인 개발** | **개인 프로젝트** | **UE5.4 · C++ · Slate** |
| 구현 및 최종 적용 기록 기준 | | Codex 활용 · 계획·구현·리뷰·검증 | Asset Registry 기반 Editor Plugin |

## 이 포트폴리오에서 보여주는 경험

### 01 AI 협업 개발 체계

초기 작업 규칙을 정하고, 기능별
계획과 검증 기록을 연결했습니다.

사전 준비·AI 리뷰 (11·15페이지) →

### 02 문제 해결과 검증

입력 오류를 수정하고, 미사용 후보
탐색 범위를 목적에 맞게 바꿨습니다.

대표 문제 해결·검증 (5–10페이지) →

### 03 플러그인 구현

참조 조회·에셋 이동·CSV 저장을
에디터 사용 흐름으로 연결했습니다.

처리 구조·CSV 저장 (12–14페이지) →

### 사용자 작업

개발 목표·작업 단위·문서 운영 기준 제시
기능 판단, 오류 확인, 검증 자료 제공

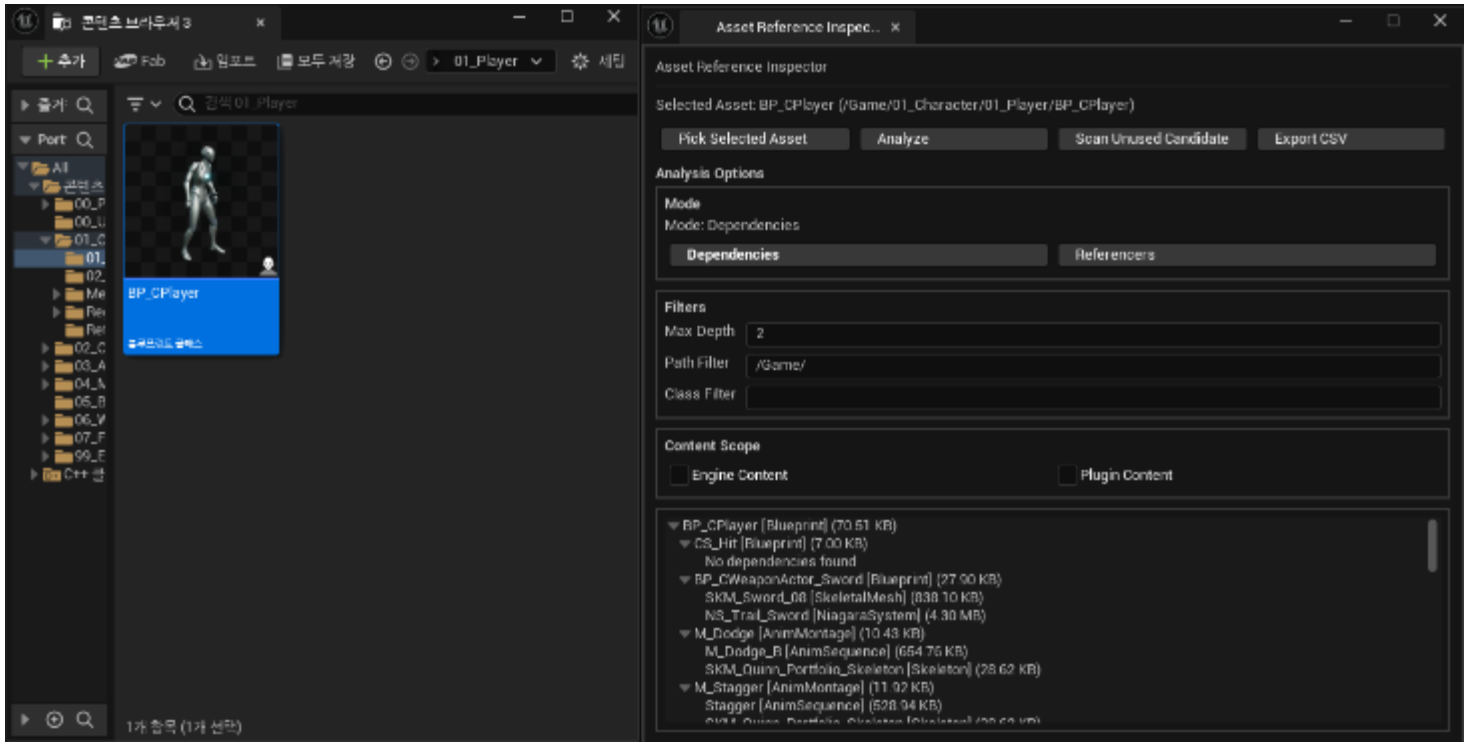
### Codex 작업

계획·문서 구체화와 코드 구현·수정
빌드 실행, 자동 리뷰와 기록 정리

각 사례의 작업은 개발 당시 기록을 기준으로 작성했습니다. 2026.10 추가 QA와 미검증 범위는 개발 당시 기록과 구분합니다.

# 에셋 참조 관계 확인과 결과 저장

선택한 에셋이 사용하는 대상과, 그 에셋을 사용하는 곳을 트리로 조회합니다.
결과 노드에서 원본 에셋으로 이동하고, 현재 분석 결과를 CSV로 저장합니다.




Dependencies: 선택 에셋이 사용하는 대상 / Referencers: 선택 에셋을 사용하는 곳

## 01 분석할 에셋 선택

콘텐츠 브라우저에서 에셋 선택
Pick Selected Asset으로 가져오기

## 02 관계 조회와 원본 이동

방향·깊이·필터를 정하고 Analyze
결과 노드 더블클릭으로 에셋 이동

## 03 분석 결과 저장

Export CSV로 현재 관계 트리 저장
에셋 이름·부모·깊이·모드 기록

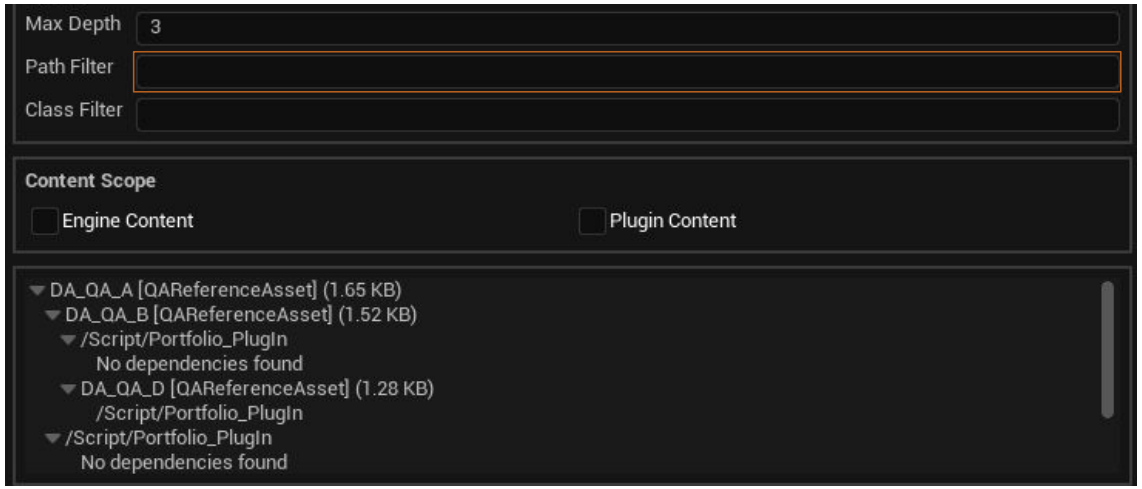
개발 당시 적용 기록 ↗

실제 프로젝트 원본 CSV ↗

# 문제: Enter 확정 뒤 경로가 지워져 필터가 풀림

특정 폴더만 조회하려고 경로를 입력했지만, Enter로 확정하면 필터가 풀렸습니다.

## 수정 전




**① Path Filter가 빈 값으로 바뀜**

**② CSV 10행 · 지정 밖 항목 포함**

## 수정 후




**① 입력한 BranchB 경로를 유지**

**② CSV 2행 · 시작 에셋 A + 필터에 일치한 B**

시작 에셋 A는 필터와 관계없이 포함

추가 QA · 2026.10.21

비교 조건: 이전 입력 처리 코드만 재구성 · 과거 전체 빌드 비교가 아님

이전 처리 CSV ↗

수정 후 CSV ↗

# 해결: Enter를 누르면 사라지던 필터 경로 유지

Enter 확정 직후 호출된 OnCleared 처리에서 저장 경로를 빈 값으로 덮어썼습니다.
OnCleared에서는 저장 경로를 변경하지 않고, Enter·클릭 확정 시 경로를 저장하도록 수정했습니다.

| | ① Enter로 확정<br>호출 사유: OnEnter | ② 후속 호출<br>호출 사유: OnCleared | ③ 분석 결과 |
|---|---|---|---|
| 수정 전 | 입력 경로 저장 | 빈 값으로 덮어씀<br>저장 경로 삭제 | 필터가 풀림<br>지정 밖 항목까지 조회 |
| 수정 후 | 입력 경로 저장 | 저장 건너뛰기<br>입력 경로 유지 | 필터 적용 유지<br>지정 범위로 조회 |

**사용자 제공 로그**

```
CommitType=1, Text='/Game/'
CommitType=3, Text='/Game/'
```

로그·역할 기록 원문 ↗

**수정 전 코드**

```
CommitType == ETextCommit::OnCleared
    ? FString() : InText.ToString();
```

수정 전 코드 원문 ↗

**수정 후 코드**

```
if (CommitType == ETextCommit::OnCleared)
    { return; }
```

수정 후 코드 원문 ↗

**사용자 작업**

오류 발견·초기 설명 재검토 요청
조작별 로그 제공

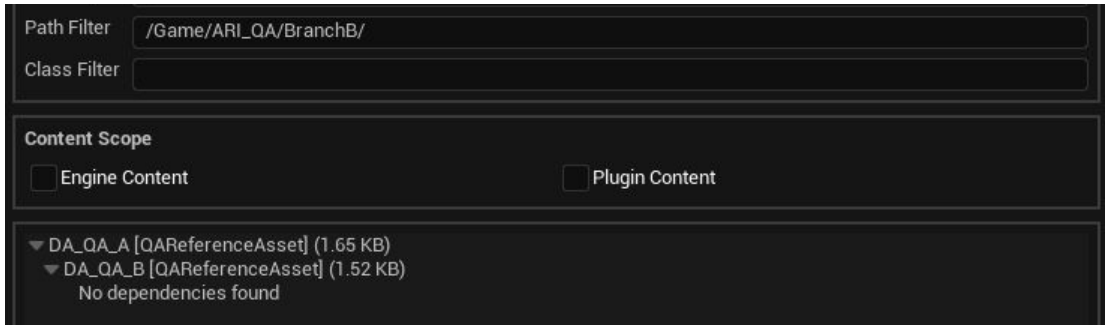
**Codex 작업**

이벤트 로그 추가·원인 분석
OnCleared 처리 수정

# 검증: Enter 확정 경로 유지와 취소·해제 동작 확인

Enter 확정 후 BranchB 경로와 CSV 2행을 확인했습니다. 클릭 확정·Escape 취소·필터 해제도 대조했습니다.

## ① B 폴더 경로 입력 → Enter → 분석




Enter 확정 후 실제 화면 ↗

## ② CSV에도 필터 결과 기록

| AssetName | Path |
|---|---|
| DA_QA_A | /Game/ARI_QA/Root |
| DA_QA_B | /Game/ARI_QA/BranchB |

원본 CSV 발췌 · 2행 모두 표시, 열 2개 선택
A는 필터와 관계없이 포함되는 시작 에셋
Enter 결과 CSV 원본 ↗

## 함께 확인한 기존 입력 동작

| 조작 | 확인된 결과 | 근거 | |
|---|---|---|---|
| C 폴더 경로 입력 → 다른 곳을 클릭<br>입력 확정 | C 경로 확정 · A와 C 출력 | 화면 ↗ | CSV ↗ |
| C 경로 확정 → B 경로로 편집 중<br>Escape | 편집 중인 B 입력 취소 → C 경로 복원<br>CSV도 기존 C 결과와 동일 | 화면 ↗ | CSV ↗ |
| 입력란을 비운 뒤 Enter | 필터 해제 · 분석 결과 10행 | 화면 ↗ | CSV ↗ |

추가 QA · 2026.10.21 · Codex 실행 · 별도 UE5.4 Data Asset 환경
비교 범위: Enter는 이전 입력 처리와 비교 · 클릭·Escape·빈 입력은 수정 후 동작만 확인 →
개발 당시: 사용자 Escape 복원 확인 기록 ↗

미검증: 과거 전체 빌드·다른 UE 버전

추가 QA 화면·영상·이벤트 로그 ↗

# 문제: 미사용 후보를 찾는데 일부 에셋만 검사

목적은 프로젝트에서 사용되지 않는 것으로 보이는 에셋을 찾는 것이었습니다.
하지만 초기 구현은 에셋 하나를 선택한 뒤, 그 에셋의 참조 관계 목록 안에서만 후보를 찾았습니다.

## 기존 | 선택 에셋의 관계만 검사

**프로젝트 콘텐츠의 에셋들**

**초기 검사 범위**

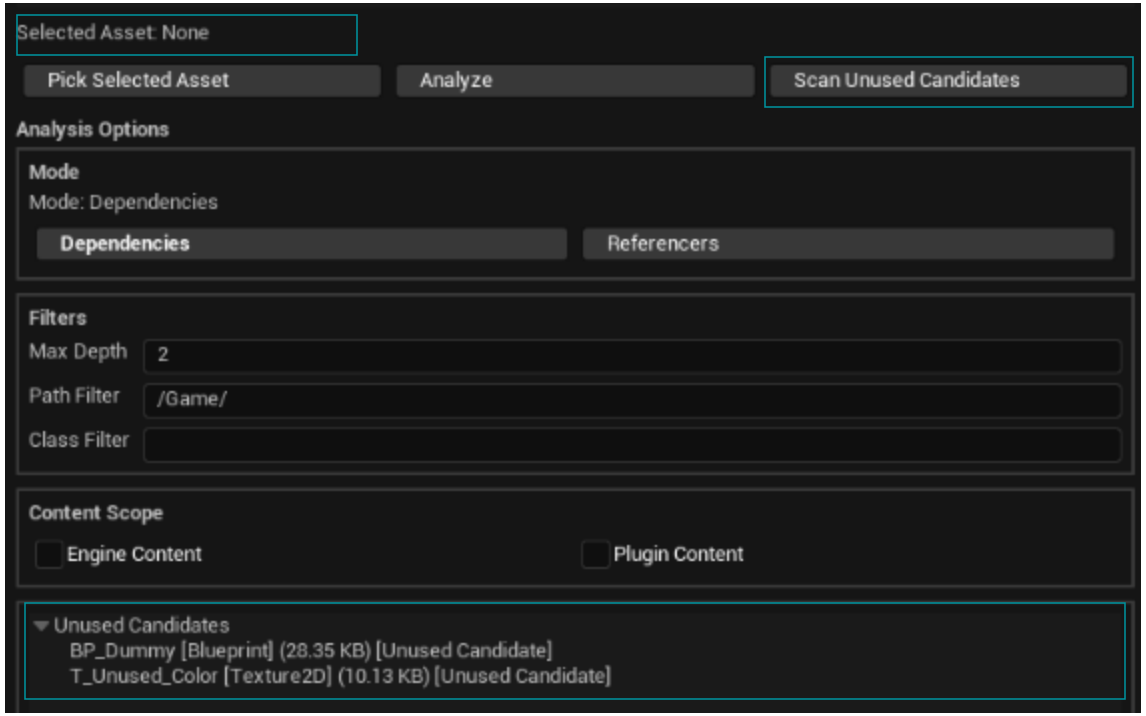
선택한 에셋 ➡ 사용하는 에셋 ➡ 그다음 에셋

**목록에 없는 에셋**

이 범위 밖 에셋은 검사하지 않음

당시 코드로 재구성한 검사 범위 예시

## 변경 | 프로젝트 콘텐츠의 미사용 후보 탐색




초기 구현·변경 기록 ↗

개발 당시 원본 화면 ↗

# 해결: 프로젝트 콘텐츠 전체에서 미사용 후보 선별

선택한 에셋의 관계 목록을 검사하는 대신, 프로젝트 콘텐츠 전체를 모아 후보를 찾도록 바꿨습니다.
기존 관계 조회는 유지하고, 에셋을 선택하지 않아도 실행할 수 있는 탐색 기능으로 분리했습니다.

| | ① 검사 대상 확대 | ② 후보 판정 유지 | ③ 후보 표시 방식 변경 |
|---|---|---|---|
| 초기 | **선택한 에셋의 관계 목록만 검사** | 다른 에셋의 참조가 없는 항목을 후보로 분류 | **참조 관계 트리 안에 미사용 후보 표시** |
| 변경 | **프로젝트 콘텐츠 전체를 검사** | 다른 에셋의 참조가 없는 항목을 후보로 분류 | **미사용 후보만 모아 전용 목록으로 표시** |

구현 범위와 근거

**프로젝트 콘텐츠(/Game) 및 하위 폴더 조회**

검사 범위 구현 ↗
후보 판정 코드·변경 기록 ↗

## 사용자 작업

콘텐츠 전체 검사로 전환 제안·계획 수정 요청
화면 제공·후보 이동 확인

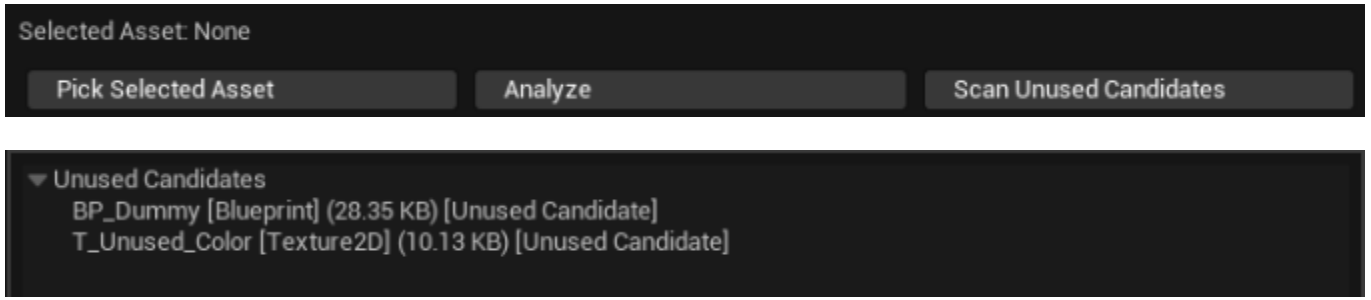
## Codex 작업

계획 재작성·전체 스캔·필터·UI 구현
빌드·검증 기록 정리

# 검증: 에셋 선택 없이 미사용 후보 조회

에셋 선택 없이 후보 2개를 조회하고, 참조받는 에셋의 제외를 확인했습니다.

## 실행 화면 | 선택 상태와 후보 목록




개발 당시 원본 화면 ↗

## 후보 탐색 결과

| | |
|---|---|
| 조회 결과 | **후보 2개 표시**<br>실제 화면 ↗ |
| 제외 확인 | **참조받는 에셋 제외**<br>사용자 확인 기록 ↗ |

## 조회·탐색 편의 기능 확인

| 조건 또는 조작 | 확인 결과 | 근거 |
|---|---|---|
| 특정 폴더만 조회 | T_Unused_Color 1개 | 실제 화면 ↗ |
| 텍스처만 조회 | Texture2D 후보 1개 | 실제 화면 ↗ |
| 일치 항목 없는 경로 조회 | 결과 없음 안내 | 실제 화면 ↗ |
| 후보 더블클릭 | 콘텐츠 브라우저에서 원본 에셋 위치로 이동 | 사용자 확인 기록 ↗ |

개발 당시 검증 · 2026.07 · UE5.4

전체 검증 범위 →

**후보 표시는 삭제 안전성을 보장하지 않습니다.**

미확인: Soft Reference·Asset Manager·동적 로딩 사용 여부
미측정: 대규모 프로젝트 성능

# AI와 함께한 개발 사전 준비

개발 범위와 우선순위를 정하고, 계획 문서와 AI 작업 지침을 마련한 뒤 구현을 시작했습니다.

## ① 개발 범위·우선순위 결정

**범위**

게임 전용 코드와 분리된
독립 에디터 플러그인

**우선순위**

Tree View 우선 · Node Graph 후순위

초기 MVP 범위 원문 ↗

## ② 계획·작업 지침 마련

**개발 계획**

계획·MVP 범위 문서로
구현 범위와 완료 조건 정리

**AI 작업 지침**

AGENTS · Codex 초기 지침

계획·초기 지침 커밋 ↗

## ③ 구현 착수

**플러그인 골격 구현**

플러그인 설정·빌드 구성
Editor 모듈 생성

골격 구현 커밋 ↗

**개발 중 보완** | 기능별 계획·검증·문서 갱신 절차 추가

기능 작업 기준 ↗

### 사용자 작업

구현 전 계획·AGENTS 설정을 먼저 요청
기능별 계획과 검증 가능한 작업 단위 제시

### Codex 작업

계획·초기 지침 문서 작성과 규칙 반영
요청한 기준에 따라 기능 작업 수행

# 플러그인 골격에 연결한 탐색·결과 처리 기능

Editor 모듈 골격을 바탕으로 참조 관계 분석과 트리 표시·에셋 이동·CSV 저장을 구현했습니다.
이후 미사용 후보 탐색을 추가하고, 실행한 기능의 결과를 공통 표시·저장 흐름으로 연결했습니다.

## 초기 계획 | 참조 관계를 트리로 조회

**관계 분석**
에셋 선택 · 방향·깊이·필터

→

**선택 에셋에서 관계 탐색**
사용하는 대상과 사용되는 곳 조회

## 이후 확장 | 프로젝트의 미사용 후보 탐색

**미사용 후보 탐색**
선택 없이 실행 · 폴더·종류

→

**프로젝트 콘텐츠에서 후보 탐색**
다른 에셋이 참조하지 않는 대상 조회

**실행한 기능의 결과로 갱신**

| | |
|---|---|
| 관계 분석 실행 | 참조 관계 트리 |
| 후보 탐색 실행 | 미사용 후보 전용 목록 |

실행할 때마다 이전 결과를 새 결과로 교체

**목록 표시**
참조 관계 트리 또는 후보 목록

→

**원본 에셋 위치로 이동**
표시된 결과 항목을 더블클릭

**CSV 저장**
현재 결과와 해당 분석 모드를 저장

### 구현 위치와 근거

**입력·탐색·표시: Slate 위젯**
Asset Registry · STreeView · RefreshTree
관계·후보 탐색과 UI 구현 ↗

**결과·분석 모드 보관**
TreeRootItems · CurrentTreeMode

**필터·CSV 처리: 별도 헬퍼**
CSV Exporter에 현재 결과·모드 전달
CSV 형식과 저장 책임 ↗

# 참조 탐색에서 공유 관계 유지와 순환 탐색 종료

여러 에셋이 같은 대상을 참조하는 관계는 각 경로에 표시해야 합니다.
같은 탐색 경로에서 다시 만난 에셋은 순환으로 표시하고, 그 아래 탐색을 멈추도록 구현했습니다.

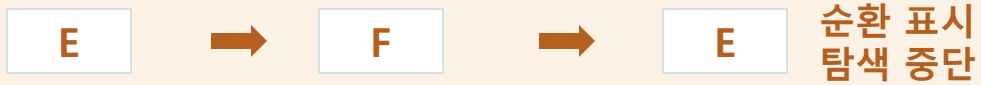
## 공유·순환 관계의 처리 기준

### 공유 | 서로 다른 참조 관계를 모두 표시

B와 C가 같은 D를 참조하므로 두 관계를 유지




### 순환 | 같은 경로에서 지난 에셋을 다시 만남




개발 당시 Blueprint 방어 전후 화면 ↗

## 검증 | 공유·순환 처리 결과

추가 QA · 2026.10.21 · 별도 Data Asset

| 검증 조건 | 기대 동작 | CSV 확인 결과 |
|---|---|---|
| B·C가 같은 D를 참조 | 두 참조 관계 모두 유지 | B→D·C→D 각각 기록 |
| E→F→E 최대 깊이 3 | 되돌아온 E 표시 하위 탐색 중단 | 깊이 2에서 다시 만난 E를 순환으로 표시 최대 깊이 3에 도달 전 종료 |

CSV: IsCircular=true · 순환 E 이후 행 없음

공유 참조 CSV ↗ 순환 참조 CSV ↗

## 사용자 작업

말단 관계 보존 판단·순환 감지 상태 요구
방어 전후 비교 제안과 실제 화면 확보

## Codex 작업

CurrentPath 기반 재귀 제어 구현
순환 노드 상태와 표시 코드 추가

추가 QA: Codex의 별도 Data Asset hard reference. 당시 Blueprint·실제 프로젝트 재실행과 크래시·성능 실측은 미수행.

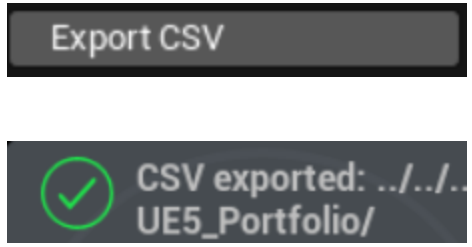
# 화면의 분석 결과를 CSV 파일로 저장

플러그인에서 확인한 참조 관계를 외부에서도 검토할 수 있도록 파일로 남겼습니다.

## ① 실제 분석 화면




## ② CSV 저장




## ③ 실제 CSV 결과

| AssetName | Depth | Mode |
|---|---|---|
| BP_Dummy | 0 | Dependencies |
| M_Dummy | 1 | Dependencies |
| T_Dummy_Color | 2 | Dependencies |

개발 당시 화면·CSV · 오른쪽 표는 일부 열 발췌

## CSV 저장 방식

**저장 대상**
실제 에셋만 저장(안내·그룹 제외)

**처리 구조**
별도 Exporter 호출

**저장 정보**
부모·깊이·모드·순환 여부 저장

원본 화면 ↗ CSV 원본 ↗ 작업 계획 초안 ↗ 구현·검증 기록 ↗

## 사용자 작업

CSV 결과 파일 3개와 화면 5장 제공
직렬화 규칙과 리뷰 범위 검토

## Codex 작업

CSV Exporter와 UI 연결 구현
계획·설계·검증 기록 작성과 빌드 실행

# CSV 개발에 활용한 AI 코드 리뷰

CSV 저장 기능 구현 후 PR #59에서 Codex 자동 리뷰를 활용했습니다.
리뷰에서 지적한 사용 조건을 수정에 반영하고, 빌드 확인과 변경 기록으로 연결했습니다.

## 개발에 적용한 리뷰 흐름

### 01 구현 코드 리뷰

CSV 저장 기능 PR #59
Codex 자동 리뷰 실행

➡

### 02 지적 내용 검토

리뷰가 지적한 사용 조건과
관련 저장 코드 검토

➡

### 03 코드 수정·빌드 확인

지적 사항 수정 반영·빌드 확인
변경 내용과 리뷰 대응 기록

## 적용 사례 | 재분석 없이 방향을 바꿀 때 CSV 모드 불일치

| | |
|---|---|
| **발견한 오류** | 재분석 없이 방향 버튼만 바꾸면, 화면의 결과는 그대로인데<br>CSV의 분석 모드만 바뀌었습니다. |
| **반영한 수정** | 현재 버튼 선택값 대신, 화면의 결과를 생성할 때 사용한<br>분석 모드를 저장합니다. |
| **추가 QA** | 방향 버튼만 변경 → 기존 결과와 CSV 유지<br>다시 분석 → 화면 결과와 CSV 모드 함께 갱신 (2026.10.21) |

자동 리뷰 원문 ↗ | 수정 코드 ↗ | 추가 QA·원본 CSV ↗ | 문서 운영 기준 ↗

### 사용자 작업

리뷰 범위 확인·Live Coding 해제 등 환경 대응
수정 후 커밋·리뷰 대응 절차 진행

### Codex 작업

자동 리뷰로 결함 발견·CSV 모드 수정·빌드
리뷰 대응과 계획·검증 기록 정리

개발 당시: 수정 코드 빌드 확인 · 추가 QA: Codex 실행 / 별도 Data Asset · 과거 전체 빌드 비교 아님